Міністерство освіти і науки України
Харківський національний університет радіоелектроніки

Кафедра штучного інтелекту

Дисципліна «Обчислювальний інтелект»

Лабораторна робота №1

«Згорткові нейронні мережі для обробки нестандартних зображень»

Виконали:
ст. гр. ДМм-26-1
Зелінський В.С.
Лавриненко С.Р.
Спащенко Р.Т.

Прийняв:
Малєєв Д.В

Харків – 2026

Реалізація згорткової нейронної мережі (CNN) для класифікації зображень з нестандартними розмірами

Мета роботи: Необхідно розробити згорткову нейронну мережу для класифікації зображень з нестандартними розмірами (наприклад, супутникові знімки або зображення природи). Для цього використайте датасет Kaggle Natural Scenes Dataset. Це завдання допоможе вам на практиці зрозуміти роботу CNN з обробкою зображень з різною роздільною здатністю.

Хід виконання роботи

1 Датасет та програмне середовище

Для експериментів використано відкритий датасет Flowers Recognition (Kaggle, alxmamaev/flowers-recognition): 4317 кольорових фотографій квітів п’яти класів – daisy (ромашка), dandelion (кульбаба), rose (троянда), sunflower (соняшник), tulip (тюльпан).

2 Розвідувальний аналіз даних (EDA)

Розподіл зображень за класами наведено в таблиці 1. Класи майже збалансовані: відношення розмірів найбільшого та найменшого класів становить 1,44, нормована ентропія розподілу 0,9932 (1,0 відповідає ідеальному балансу).

Таблиця 1 – Розподіл зображень за класами

Клас	Кількість	Частка, %
daisy	764	17,70
dandelion	1052	24,37
rose	784	18,16
sunflower	733	16,98
tulip	984	22,79
Усього	4317	100,00

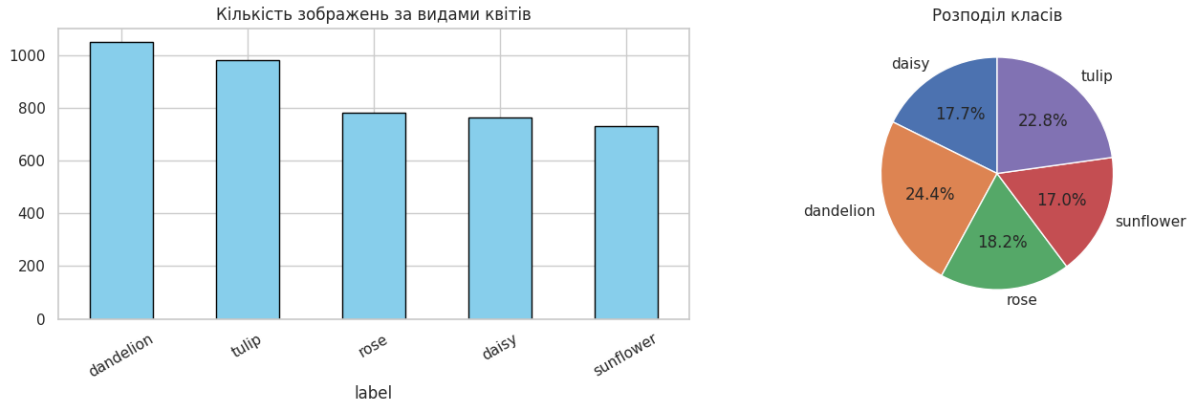


Рисунок 1 – Кількість зображень та розподіл за класами

Зображення мають суттєво різні розміри та пропорції (таблиця 2): ширина становить 134–1024 пікселів, висота 80–442, медіанний розмір 320×240. Найпоширеніші роздільні здатності: 320×240 (13,99 %), 500×333 (13,88 %), 240×159 (8,76 %). Співвідношення сторін коливається від 0,56 до 5,48; три зображення мають меншу сторону менше 100 px, 31 зображення має співвідношення сторін понад 2 або менше 0,5. Це обґрунтовує необхідність приведення зображень до єдиного розміру.

Таблиця 2 – Описова статистика метаданих зображень

Показник	Середнє	Std	Мін.	Медіана	Макс.
Ширина, px	338,38	119,07	134	320	1024
Висота, px	253,07	61,56	80	240	442
Співвідношення W/H	1,33	0,31	0,56	1,44	5,48
Розмір файлу, КБ	54,10	44,91	5,39	36,54	275,34

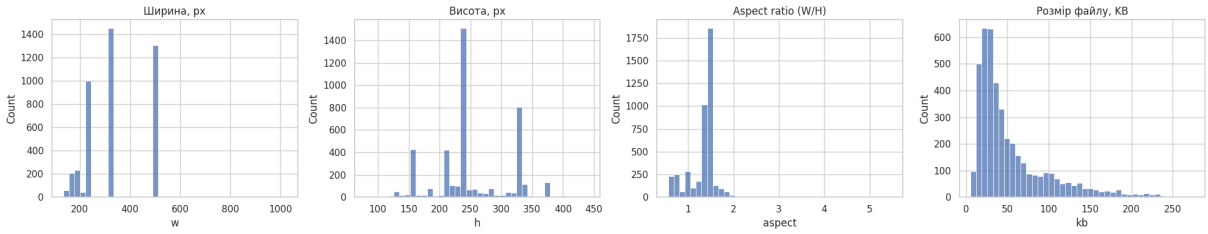


Рисунок 2 – Розподіли ширини, висоти, співвідношення сторін та розміру файлу

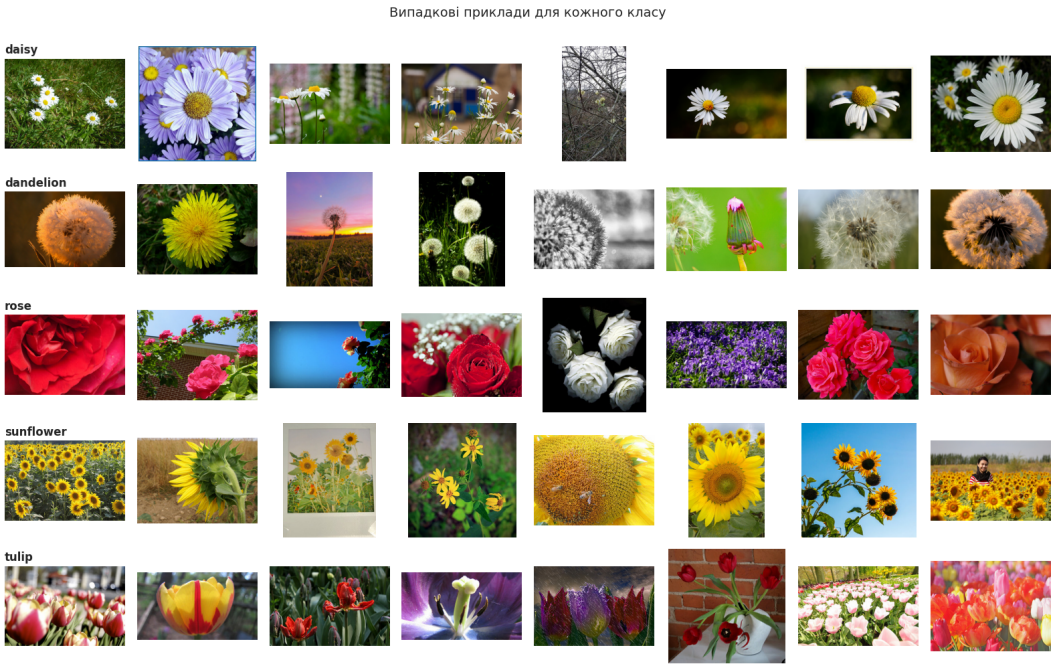


Рисунок 3 – Випадкові приклади зображень кожного класу

Піксельна статистика (стратифікована вибірка до 300 зображень на клас, стиснених до 64×64) показує відмінності між класами за кольором (таблиця 3): соняшники найяскравіші (0,463) та найнасиченіші (0,605) і мають найвищу інтенсивність червоного каналу, у троянд найнижча інтенсивність зеленого каналу (0,366). Водночас жовті квіти (кульбаба, соняшник, ромашка) подібні за кольором, що ускладнює класифікацію.

Таблиця 3 – Середні піксельні характеристики за класами (шкала 0–1)

Клас	R	G	B	Яскравість	Контраст	Насиченість
daisy	0,415	0,424	0,330	0,411	0,223	0,442
dandelion	0,427	0,433	0,283	0,414	0,194	0,482
rose	0,483	0,366	0,325	0,396	0,199	0,455
sunflower	0,500	0,481	0,272	0,463	0,204	0,605
tulip	0,484	0,401	0,312	0,415	0,198	0,491

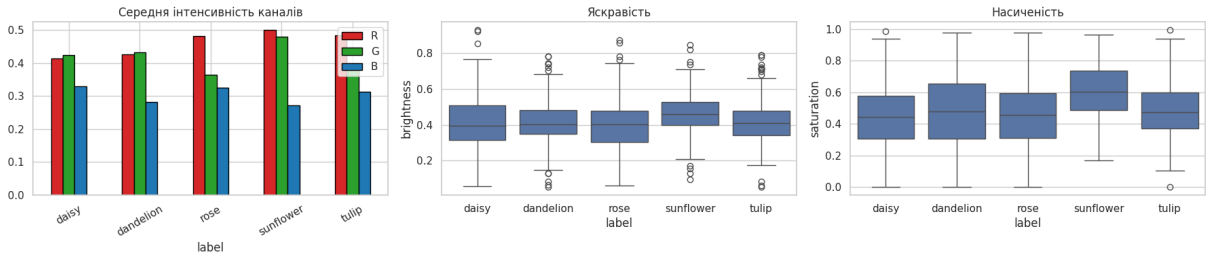


Рисунок 4 – Середня інтенсивність каналів, яскравість та насиченість за класами

3 Якість даних та підготовка вибірок

Перевірка за хешем MD5 виявила 6 груп точних дублікатів (12 файлів). У 4 групах те саме зображення міститься одночасно в класах rose та tulip (конфліктні мітки), ще у 2 групах дублікати належать одному класу (sunflower). Групи з конфліктними мітками видалено повністю, з решти дублікатів залишено по одному файлу. Після очищення залишилося 4307 зображень із 4317.

Очищений набір розбито стратифіковано на навчальну (70 %), валідаційну (15 %) та тестову (15 %) вибірки (таблиця 4). Тестова вибірка не використовувалась ні для навчання, ні для підбору епохи зупинки.

Таблиця 4 – Розбиття очищеного датасету

Клас	Train	Val	Test	Усього
daisy	535	114	115	764
dandelion	736	158	158	1052
rose	546	117	117	780
sunflower	511	110	110	731
tulip	686	147	147	980
Усього	3014	646	647	4307

3.4 Приведення зображень до розміру 196×196

Реалізовано два способи приведення до єдиного розміру 196×196:

– Stretch – зображення масштабується по осях незалежно та повністю заповнює кадр; пропорції спотворюються;
– Pad – довша сторона масштабується до 196 px, пропорції зберігаються, решта кадру заповнюється нулями (чорні поля), зображення розташовується по центру.

Піксельні значення нормалізуються діленням на 255 (rescale = 1/255). Частка повністю чорних пікселів у навчальній вибірці становить 29,1 % для варіанта Pad проти 1,21 % для Stretch, тобто Pad подає мережі значну частку неінформативних пікселів.

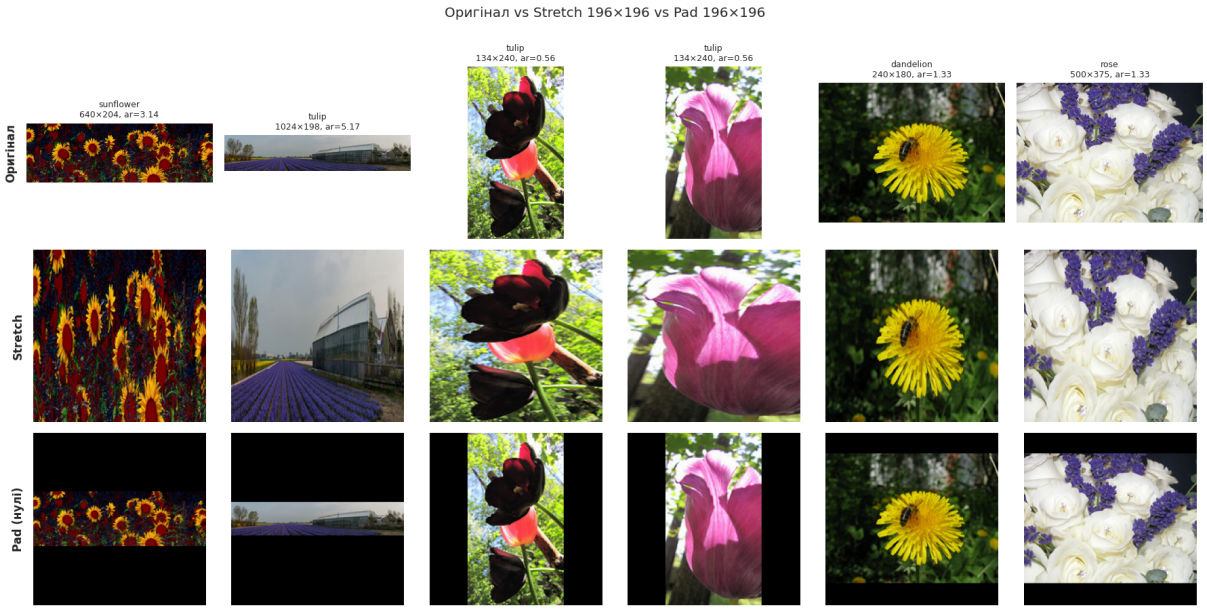


Рисунок 5 – Оригінал, Stretch та Pad 196×196 для зображень із різним співвідношенням сторін

3.5 Аугментація зображень

Для розширення навчальної вибірки використано ImageDataGenerator (Keras). Аугментація застосовується лише до навчальних даних; валідаційні та тестові зображення тільки нормалізуються. Параметри однакові для обох способів ресайзу: поворот до 25°, зсув по ширині та висоті до 10 %, зсув (shear) 0,1, масштабування 0,15, горизонтальне віддзеркалення, зміна яскравості в діапазоні 0,8–1,2; нові області після перетворень заповнюються нулями (fill_mode = constant, cval = 0), що узгоджується з заповненням у варіанті Pad.



Рисунок 6 – Приклади аугментації одного зображення (Stretch та Pad)

3.6 Архітектура CNN

Мережа складається з п’яти блоків Conv2D (ядро 3×3, padding = same, ReLU) - MaxPooling2D (2×2) із кількістю фільтрів 16, 32, 64, 128, 128. Після двох останніх блоків додано Dropout (0,25). Далі йдуть Flatten, повнозв’язний шар Dense(256, ReLU), Dropout (0,5) та вихідний Dense(5, softmax). Загальна кількість параметрів – 1 426 213 (5,44 МБ). Функція втрат – категоріальна крос-ентропія.

Таблиця 5 – Архітектура моделі (базова конфігурація)

Шар	Вихідна форма	Параметрів
Input	196×196×3	0
Conv2D(16) + MaxPool	98×98×16	448
Conv2D(32) + MaxPool	49×49×32	4 640
Conv2D(64) + MaxPool	24×24×64	18 496
Conv2D(128) + MaxPool + Dropout(0,25)	12×12×128	73 856
Conv2D(128) + MaxPool + Dropout(0,25)	6×6×128	147 584
Flatten	4608	0
Dense(256, ReLU) + Dropout(0,5)	256	1 179 904
Dense(5, softmax)	5	1 285
Разом		**1 426 213**

3.7 Умови навчання

Усі моделі навчалися за однакових умов: розмір пакета 32, максимум 100 епох, ті самі розбиття та seed. Для оптимізатора Adam початковий крок навчання $1 \cdot 10^{-3}$; використано EarlyStopping за валідаційною втратою (patience = 6, відновлення найкращих ваг) та ReduceLROnPlateau (коефіцієнт 0,5, patience = 3, мінімум $1 \cdot 10^{-5}$). Через спрацювання EarlyStopping навчання зупинялося раніше за 100-ту епоху (таблиця 6).

3.8 Порівняння Stretch та Pad (базова модель: Adam + Dropout)

Результати базової моделі на тестовій вибірці (647 зображень) наведено в таблиці 6.

Таблиця 6 – Результати базової моделі (Adam + Dropout)

Варіант	Test acc	Macro-F1	Test loss	Найкраща епоха	Епох виконано	Час, хв
Stretch	0,8099	0,8078	0,5159	28	34	15,5
Pad	0,7743	0,7710	0,6084	22	28	12,7

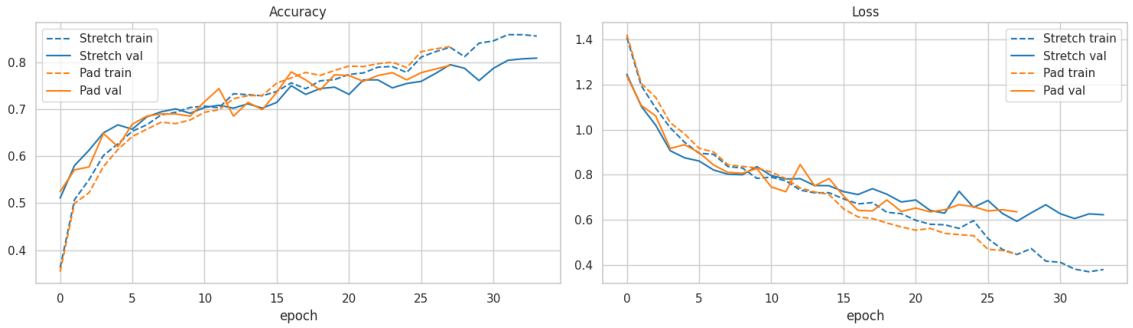


Рисунок 7 – Криві навчання базової моделі: accuracy та loss (пунктир – train, суцільна – val)

Таблиця 7 – Метрики за класами на тесті (Adam + Dropout)

Клас	Stretch: precision	Stretch: recall	Stretch: F1	Pad: precision	Pad: recall	Pad: F1
daisy	0,809	0,774	0,791	0,727	0,809	0,765
dandelion	0,817	0,873	0,844	0,784	0,804	0,794
rose	0,750	0,744	0,747	0,766	0,615	0,682
sunflower	0,826	0,909	0,866	0,779	0,927	0,846
tulip	0,840	0,748	0,791	0,811	0,728	0,767

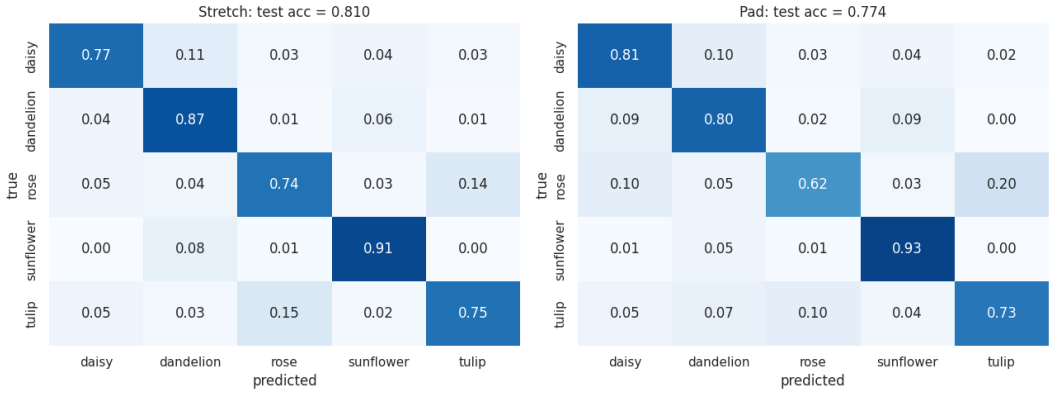


Рисунок 8 – Нормовані матриці помилок на тестовій вибірці (Stretch та Pad)

Базова модель із Stretch досягла точності 80,99 % проти 77,43 % для Pad. Найкраще розпізнаються соняшники (F1 = 0,866 для Stretch) та кульбаби; найскладніший клас – троянда (F1 = 0,747 та 0,682), чий recall у варіанті Pad падає до 0,615. Парне порівняння на тих самих тестових зображеннях: різниця Pad − Stretch становить −3,55 в.п., 95 % bootstrap-

інтервал [−6,34; −0,77], точний тест Мак-Немара p = 0,017 (Stretch правильний, а Pad помилився на 54 зображеннях; зворотна ситуація – на 31).

3.9 Регуляризація та оптимізатор: L2 замість Dropout, SGD замість Adam

Для покращення моделі досліджено ще дві конфігурації:

– Adam + L2 – шари Dropout прибрано, до всіх Conv2D та Dense додано L2-регуляризацію ($\lambda = 5 \cdot 10^{-4}$);

– SGD + Dropout – оптимізатор SGD із моментом 0,9 (Nesterov), обмеженням норми градієнта (clipnorm = 5) та косинусним згасанням кроку навчання до 1 % від початкового; EarlyStopping із patience = 8.

Початковий крок навчання для SGD визначено тестом діапазону LR (LR range test): за 200 кроків LR експоненційно зростав від 10^{-4} до 1, початковий крок вибирався на порядок нижче за точку мінімуму згладженої втрати. Для порівнянності валідаційної втрати між конфігураціями моніторилась чиста крос-ентропія без L2-штрафу.

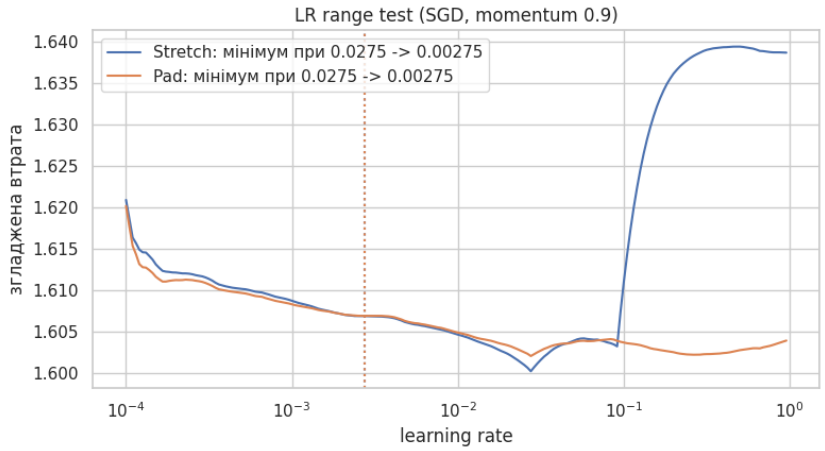


Рисунок 9 – Тест діапазону LR для SGD (momentum 0,9)

Таблиця 8 – Порівняння конфігурацій та способів ресайзу на тесті (seed = 42)

Конфігурація	Ресайз	Test acc	Macro-F1	Test CE	Найкр. епоха	Епох	Час, хв
Adam + Dropout	Stretch	0,8099	0,8078	0,5159	28	34	15,5
Adam + Dropout	Pad	0,7743	0,7710	0,6084	22	28	12,7
Adam + L2	Stretch	0,8053	0,8045	0,5809	42	48	21,7
Adam + L2	Pad	0,7805	0,7773	0,6408	34	40	18,1
SGD + Dropout	Stretch	0,7666	0,7617	0,6142	63	71	32,1
SGD + Dropout	Pad	0,7743	0,7706	0,6004	88	96	43,2

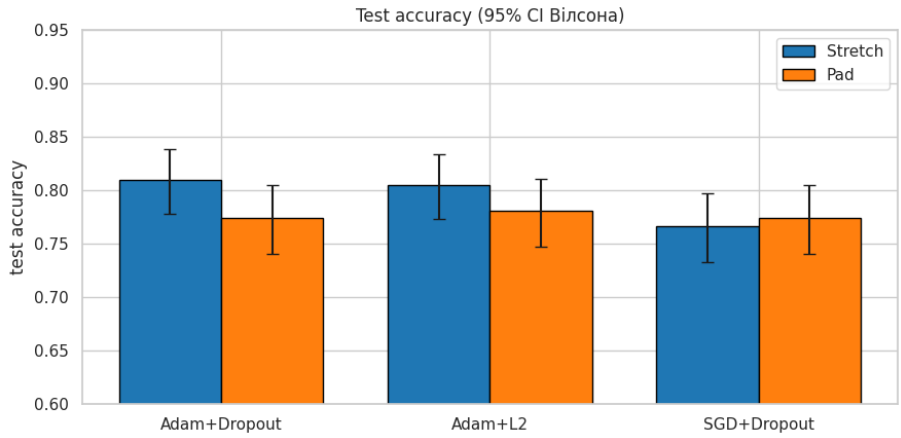


Рисунок 10 – Test accuracy конфігурацій із 95 % довірчими інтервалами Вілсона

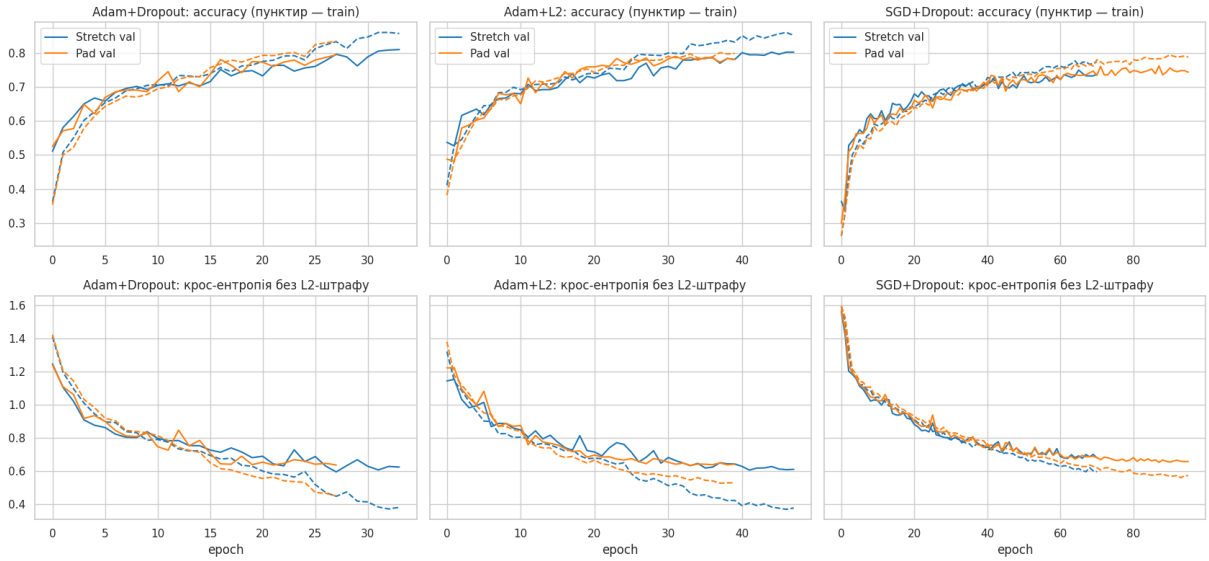


Рисунок 11 – Криві навчання всіх конфігурацій (accuracy та крос-ентропія)

Для кожної конфігурації виконано парне порівняння Stretch і Pad на тому самому тесті (таблиця 9). Для обох варіантів Adam Stretch точніший за Pad (для Adam + Dropout різниця значуща, p = 0,017; для Adam + L2 – на межі значущості, p = 0,081). У конфігурації з SGD різниця незначна й має протилежний знак (+0,77 в.п., p = 0,52). Мінімальна різниця, яку такий тест здатен виявити з значущістю 80 %, становить 2,7–4,0 в.п., тому дрібніші відмінності не можна вважати доведеними.

Таблиця 9 – Парне порівняння Pad − Stretch усередині конфігурацій

Конфігурація	Acc Stretch	Acc Pad	Різниця	95 % CI	p (Мак-Немар)	MDE 80 %
Adam + Dropout	0,8099	0,7743	−0,0355	[−0,063; −0,008]	0,017	0,040
Adam + L2	0,8053	0,7805	−0,0247	[−0,051; +0,002]	0,081	0,037
SGD + Dropout	0,7666	0,7743	+0,0077	[−0,011; +0,026]	0,522	0,027

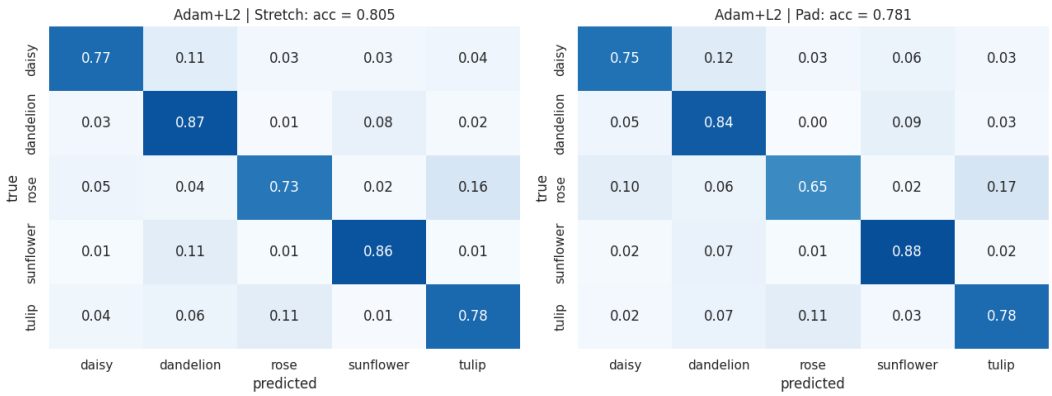


Рисунок 12 – Матриці помилок найкращої за середньою точністю конфігурації (Adam + L2)

3.10 Повтори з різними seed

Щоб відокремити ефект способу ресайзу від випадковості ініціалізації та аугментацій, базову конфігурацію (Adam + Dropout) навчено для трьох значень seed (42, 43, 44) на тих самих розбиттях (таблиця 10).

Таблиця 10 – Test accuracy базової моделі для різних seed

Seed	Stretch	Pad	Pad − Stretch
42	0,8099	0,7743	−0,0355
43	0,8068	0,7975	−0,0093
44	0,8223	0,7975	−0,0247
Середнє ± σ	0,8130 ± 0,0082	0,7898 ± 0,0134	−0,0232

У всіх трьох запусках Stretch не поступається Pad, але парний t-тест за seed ($p = 0{,}093$) при трьох спостереженнях має низьку значущість. Ансамбль із трьох моделей (усереднення ймовірностей) підвищує точність для обох способів: Stretch 0.8362, Pad 0.8192.

Висновки: Розроблено та досліджено CNN для класифікації зображень квітів п’яти класів із різною роздільною здатністю та співвідношенням сторін. Приведення до 196×196 способом Stretch у середньому дає кращу точність, ніж Pad: 80,99 % проти 77,43 %. Adam + Dropout та Adam + L2 дають близьку точність (80,99 % і 80,53 % для Stretch). Найкращий результат на тестовій вибірці серед одиночних моделей 81,0 % (Adam + Dropout, Stretch), серед ансамблів – 83,6% (три моделі, Stretch).

Додаток А. Фрагменти програмного коду

А.1 Приведення зображень до розміру 196×196

```
def resize_stretch(im, size=196):
    return np.asarray(im.convert("RGB").resize((size, size), Image.LANCZOS),
dtype=np.uint8)

def resize_pad(im, size=196):
    im = im.convert("RGB"); w, h = im.size
    s = size / max(w, h)
    nw, nh = max(1, round(w * s)), max(1, round(h * s))
    small = np.asarray(im.resize((nw, nh), Image.LANCZOS), dtype=np.uint8)
    canvas = np.zeros((size, size, 3), dtype=np.uint8)
    y0, x0 = (size - nh) // 2, (size - nw) // 2
    canvas[y0:y0 + nh, x0:x0 + nw] = small
    return canvas
```

А.2 Аугментація

```
aug_params = dict(rescale=1.0/255, rotation_range=25, width_shift_range=0.1,
                  height_shift_range=0.1, shear_range=0.1, zoom_range=0.15,
                  horizontal_flip=True, brightness_range=(0.8, 1.2),
                  fill_mode="constant", cval=0.0)
train_gen = ImageDataGenerator(**aug_params).flow(X_tr, y_tr, batch_size=32,
shuffle=True, seed=SEED)
```

А.3 Модель CNN

```
def build_cnn(n_classes=5, filters=(16, 32, 64, 128, 128), lr=1e-3):
    m = keras.Sequential(name="flower_cnn")
    m.add(layers.Input(shape=(196, 196, 3)))
    for i, f in enumerate(filters):
        m.add(layers.Conv2D(f, 3, padding="same", activation="relu"))
        m.add(layers.MaxPooling2D(2))
        if i >= len(filters) - 2:
            m.add(layers.Dropout(0.25))
    m.add(layers.Flatten())
    m.add(layers.Dense(256, activation="relu"))
    m.add(layers.Dropout(0.5))
    m.add(layers.Dense(n_classes, activation="softmax"))
    m.compile(optimizer=keras.optimizers.Adam(lr),
              loss="categorical_crossentropy", metrics=["accuracy"])
    return m
```

А.4 Навчання

```
cbs = [keras.callbacks.EarlyStopping(monitor="val_loss", patience=6,
restore_best_weights=True),
       keras.callbacks.ReduceLROnPlateau(monitor="val_loss", factor=0.5,
patience=3, min_lr=1e-5)]
model.fit(train_gen, validation_data=val_gen, epochs=100, callbacks=cbs,
verbose=2)
```